From Video to Humanoid: A Simulation Pipeline for Motion Imitation on the Unitree R1, and What Breaks It in Transfer

Shaoqin Li[1] Yi Sun[1]

[1]Applied and Embodied AI Laboratory, The City College of New York

October 16, 2026

Manuscript, simulation study. First draft.

Revised from the 2026-10-11 manuscript: adds the three-seed delay study (Section 7.4) and corrects the timing of the dance reference (Table 2, Section 7).

Abstract

Humanoid robots can now learn skills from human video, but the open tools for it are built around a handful of robots, mostly Unitree's G1 and H1. We took the Unitree R1, a 1.23 m humanoid that costs a fraction of the G1, and built the full chain from a single-camera video to a trained motion-tracking policy, in simulation. The chain strings together four existing systems: GVHMR recovers the person's motion, GMR retargets it to the robot, a BeyondMimic-style PPO policy learns to track it in mjlab or Isaac Lab, and a standalone CPU MuJoCo harness checks the policy in a second simulator. We add the R1 to each of them and release every stage. We ran two clips through it, a tennis serve and a dance, and tested the policies under the training randomization and 15 stress scenarios. Three findings stand out. First, our first sim2sim failure was a harness problem, not a physics gap: a missing floor and the wrong timestep explain all of it. Second, command delay is the dominant failure. At 40 ms of added delay, survival is 2/20 for the tennis policy and 0/20 for the dance policy, which were trained in different engines. Training with randomized delay brings both to 15/20, though in PhysX it costs robustness when there is no delay (29/30 to 17/30). Across three training seeds the gain in mjlab comes from the delay itself (15–20/20, against 2–8/20 for an equally long control run without delay). Third, joint-space error overstates the dance gap: in 27% of reference frames the retargeted motion pins the left shoulder at a joint limit, and retargeting still reports no violations. Everything here is simulation. Nothing has been run on a physical R1.

1 Introduction

Human video is the cheapest source of humanoid demonstrations there is. The recipe for using it has settled into a pipeline: recover the person's 3D motion from the video, retarget it onto the robot's kinematics, train a policy in simulation to track the retargeted reference, and test the policy in a second simulator before going near hardware [1, 2, 3, 4]. Each stage has a strong open-source implementation. The catch is that these implementations are developed and tested on a few robots, most often the Unitree G1 and H1. Pointing the pipeline at a different robot means writing code at every boundary: the robot model, the IK configuration, joint orderings and conventions, actuator limits, and an evaluation harness.

This paper does that for the Unitree R1 and reports what we learned. The R1 has 26 degrees of freedom (24 of them in the body, which is what we model), weighs about 29 kg, and sells for $5,900 [5], so it is a plausible platform for labs that cannot buy a G1. We found no published motion-imitation work on it, and as of 2026-10-11 it is not in GMR's list of supported robots.

We want to be clear up front about what is ours. None of the four core algorithms is. GVHMR, GMR, the BeyondMimic tracking recipe and PPO were written by others, and Section 3 lists exactly who, and what we changed. What we contribute is:

1. **An R1 pipeline, end to end.** R1 support for GMR, mjlab and Isaac Lab (robot model, IK configurations, tracking tasks including a randomized-delay variant), the converters between stages, a standalone sim2sim harness, and a Blackwell-GPU install recipe. Every stage was re-run from fresh clones and reproduced the recorded outputs (Section 6).
2. **A cross-engine robustness study.** Policies trained in MuJoCo-Warp and in PhysX are tested in a third setting, plain CPU MuJoCo, under the training randomization and 15 stress scenarios (Section 7).
3. **Three findings about transfer:** a sim2sim harness must match the training model's floor and timestep before any gap means anything; command delay dominates failure regardless of training engine; and joint-space error misleads on video-derived references that saturate joint limits (Section 8).

2 Related work

Learning humanoid skills from human motion. Most of this work runs on Unitree robots. H2O and OmniH2O retarget human motion to a full-size humanoid and train policies for real-time whole-body teleoperation [2, 6]. ASAP trains tracking policies for agile skills on the Unitree G1 and learns a residual action model from real data to close the sim-to-real gap [3]. KungfuBot and GMT track highly dynamic or general human motions on the G1 [7, 8], and TWIST adds a teleoperation system on top of a G1 motion-tracking controller [9]. BeyondMimic gives one tracking recipe (observations, rewards, randomization, termination) that reaches dynamic motions on hardware with shared hyperparameters [4]. Amadio and Mingo Hoffman compare design choices in such pipelines on the G1 [10]. VideoMimic goes furthest in the direction of this paper, from casual video to humanoid policies, and adds scene reconstruction [1].

Human motion recovery and retargeting. GVHMR estimates SMPL-X [11] body motion in a gravity-view frame, which removes much of the world-frame drift of earlier methods [12]. GMR retargets human motion to many humanoids with a two-stage IK and shows that retargeting quality changes downstream tracking performance [13]. OmniRetarget preserves human-object and human-scene interaction during retargeting [14]. Earlier work retargeted by fitting robot shape and optimizing joint angles [2]. For physics-based tracking of large motion libraries, PHC established the approach in character animation [15].

Sim2sim and transfer. Humanoid-Gym made train-in-one-simulator, verify-in-another an explicit step before hardware [16]. PolySim goes further and trains across several engines at once [17]. Domain randomization is the standard defence against the gaps such checks expose [18, 19], and actuator and latency modelling has long been known to matter for legged transfer [20]. We take sim2sim as given and ask what a second simulator actually tells you, on a robot nobody has tested it on.

The R1. The R1 is new (2025) and cheap. We are not aware of a published imitation or locomotion study on it. Not to be confused with the Galaxea R1 Pro, a wheeled robot that appears in GMR's list. Our own earlier R1 walking experiments (internal lab notes, unpublished [21]) found that a policy passing a sim2sim check fell within seconds under a real-time harness, which is what pushed us toward the delay question in this paper.

3 Where everything comes from

This section answers three questions a reader should be able to ask: which ideas are borrowed, which parts are ours, and where the videos came from.

3.1 Algorithms and software: borrowed versus ours

Table 1: Where each piece of the pipeline comes from. "Adapted" means somebody else's system that we extended to the R1.

Piece	Source	Origin	What we did with it
Human motion recovery (video to SMPL-X)	GVHMR, Shen et al., SIGGRAPH Asia 2024 [12]. Pinned at `ee960bb`.	Borrowed	Ran unmodified, static-camera option. Wrote the install recipe that makes it work on a Blackwell GPU.
Body model	SMPL-X, Pavlakos et al., CVPR 2019 [11]	Borrowed	Not redistributed (license). Users register for it themselves.
Retargeting (SMPL-X to robot joints)	GMR, Araujo, Ze et al., 2025 [13, 22]. Pinned at `bb1bbe4`.	Adapted	GMR has no R1. We added the R1 model, meshes and IK configs for all eight GMR input formats (generated from one ratio rule, checked against the two hand-written configs), with an installer and a validator.
Ground-drift correction	Our own preprocessing step	Ours	GVHMR drifts about 15 cm vertically over the tennis clip. We fit a line to the lower envelope of foot height and subtract it on the human frames, before IK. GMR itself is untouched.
Tracking recipe (observations, rewards, randomization, termination)	BeyondMimic, Liao et al., 2025 [4]	Borrowed	Used as published, with R1 gains and torque limits.
Training framework 1 (MuJoCo-Warp)	mjlab, Zakka et al., 2026 [23]. Pinned at `c2e1e06`.	Adapted	Registered the R1 as a new robot with three tasks: tracking, tracking with randomized delay, and tracking without state estimation.
Training framework 2 (PhysX)	Isaac Lab [24] with BeyondMimic's `whole_body_tracking` [25]. Pinned at `b0542fe2d` / `cd65172`.	Adapted	Added the R1 and a delay action term. Generated the URDF from the same MJCF. Made the motion loader reorder joints by name, because PhysX orders the 24 joints differently from MuJoCo.

Piece	Source	Origin	What we did with it
RL algorithm	PPO, Schulman et al., 2017 [26], through the RSL-RL library [27] that both frameworks use	Borrowed	Used as is.
Physics	MuJoCo, Todorov et al., 2012 [28]; MuJoCo-Warp; PhysX	Borrowed	Used as is.
R1 robot model and meshes	`unitree_mujoco` (Unitree, BSD-3-Clause) [29]	Borrowed	One MJCF with 43 meshes for every stage. PD gains and torque limits taken from Unitree's deployment description.
Random command delay	A standard technique in sim-to-real work [19, 20]	Adapted	We implemented it for the R1 in both engines (0 to 50 ms per robot, redrawn every second). We do not claim the idea.
Sim2sim harness	Idea from Humanoid-Gym [16]. Code is ours.	Ours	Standalone CPU MuJoCo. Rebuilds the training model, reimplements observations, randomization and termination, adds 15 stress scenarios, and validates its first observation against Isaac Lab's (difference 0.00000).
Floor and timestep ablation	Our experiment	Ours	One-at-a-time ablation that traces the first sim2sim failure to its cause (Section 7.1).
Joint-limit saturation diagnostic	Our analysis	Ours	Shows that error on the dance reference comes from frames where the reference pins a joint at its limit (Section 8.3).

3.2 The two video clips

3.3 Does any video work?

No, and we only know part of the answer. Two clips is not enough to say where the pipeline breaks. Here is what we know and what we do not.

What both clips have in common. One person, whole body in frame including the feet, a static camera, and a clean view with no heavy occlusion.

What the tools claim beyond that. GVHMR has a visual-odometry path for moving cameras (the README describes DPVO, and a lighter option added since). We did not use it for the results in this paper. We cannot say how the ground-drift correction or the policy training behave on a handheld clip.

What should break it, but we have not tested. Several people in frame, the feet cropped out, strong occlusion, or motion where the feet leave the ground or the person works on the floor. The R1 has no head or hand joints in our model, so motions that depend on them are lost. We expect these to hurt Stage 1 or the policy, but we have not run any of them.

The motion also has to suit the robot. Retargeting always produces an answer, and it can be a poor one. On the dance reference at least one joint sits at its limit on 62% of frames, and on the tennis reference on 31% (Section 8.3). The retargeted reference is also not physically realisable by

Table 2: The two source clips. Neither was filmed by us.

	Tennis serve	Dance
Source	`docs/example_video/tennis.mp4`, shipped with the GVHMR repository [12]. GVHMR's README does not say where the clip comes from or under what license. The frames show a practice court at the US Open.	"Woman Dancing" by Antoni Shkraba, a free stock video on Pexels [30].
Used length	10.4 s, 312 frames at 30 fps	26.5 s, 662 frames at 25 fps. GMR's GVHMR loader assumes 30 fps, so the retargeted reference plays 1.2× too fast (22.1 s). Corrected 2026-10-14; see Section 7.
Camera	Static	Static
Content	One person, whole body in frame, a tennis serve (ball toss to follow-through)	One person, whole body in frame, a dance routine
Stage 1 time (RTX 5080)	40.6 s, almost all preprocessing	100.9 s, almost all preprocessing
Redistributed here?	No video. Two still frames are shown in Figure 3 for illustration.	No. The clip and its frames are not redistributed. Only the robot renders and derived motion data are.

itself: its feet sink up to 2.9 cm into the floor and peak joint speed reaches 22.5 to 24.9 rad/s against a 20 rad/s servo budget we assumed. The policy has to repair that, and the policy only did so for the two clips we trained it on.

So the safe claim is narrower than "any video": a static-camera clip of one person, whole body visible, produced a working policy in 2 of 2 attempts. Testing a moving-camera, handheld clip is the obvious next experiment (Section 10).

4 The robot

The Unitree R1 stands 1.23 m tall and weighs about 29 kg [5]. Unitree lists 26 degrees of freedom: 6 per leg, 5 per arm, 2 in the waist and 2 in the head. Our model has 24 driven joints (the head is left out) and one MJCF with 43 meshes for every stage. PD gains and torque limits come from Unitree's deployment description. Early in the project we left the torque limits out, and the policy's corrective commands grew without bound, so they are enforced everywhere. Table 3 lists the actuator parameters shared by training and evaluation.

5 The pipeline

5.1 Stage 1: video to SMPL-X (GVHMR)

We run GVHMR at the pinned commit with the static-camera option. On the RTX 5080 (Blackwell) the stock install fails in five separate ways: torch pins, no pytorch3d wheel, split CUDA headers,

Table 3: Actuators, as set in `sim2sim_r1_tennis_v4_standalone.py` and identical in the mjlab robot definition.

Joint group	K_p	K_d	Torque limit (N m)
hip pitch / roll / yaw	150	8	88
knee	200	10	139
ankle pitch / roll	60	4	50
waist roll / yaw	80	5	50 / 88
shoulder pitch / roll	60	4	25
shoulder yaw, elbow, wrist roll	40	3	25

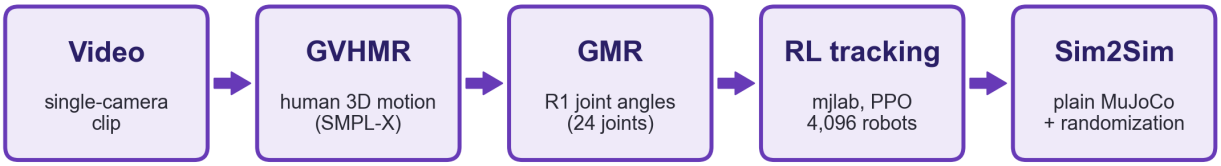


Figure 1: The five stages. Stages 1 and 2 turn a video into R1 joint angles. Stage 3 trains a tracking policy. Stage 4 tests it in a second simulator.

an undeclared dependency, and a missing `nvcc`. `SETUP.md` in the release has the recipe that works. Processing takes 40.6 s for the 10.4 s tennis clip and 100.9 s for the 26.5 s dance clip, almost all of it preprocessing (detection, 2D pose, features).

5.2 Stage 2: SMPL-X to the R1 (GMR)

The R1 goes into a stock GMR clone with one installer script. The video path uses `smplx_to_r1.json`. The ground-drift correction from Table 1 runs on the human frames before IK. IK runs at 305 to 449 frames per second on CPU and outputs root position, root orientation and 24 joint angles at 30 fps.

We checked the retargeted tennis motion with a short plausibility script. Joint-limit violations: none. Peak joint speed: 22.5 rad/s, above the 20 rad/s budget on 4 frames. Feet in ground contact on 9% of frames (27 of 312), with the lowest foot point 2.25 cm below the floor. Pelvis height range 18.5 cm. The dance motion looks similar: no violations, 24.9 rad/s peak on 4 frames, contact on 34% of frames (227 of 662), 2.87 cm penetration. A retargeted motion is a reference and not a controller, and the zero violations hide a problem we come back to in Section 8.3.

5.3 Stage 3: trajectory to tracking policy

The trajectory is replayed in mjlab to make a 50 Hz reference with full-body positions, orientations and velocities. We then train with the BeyondMimic recipe: PPO, 4,096 parallel robots, 50 Hz control over 200 Hz physics (5 ms by 4). The policy sees a 135-dimensional observation: reference joint positions and velocities (48), the torso (anchor) position and orientation relative to the reference (3 + 6), base linear and angular velocity (6), joint positions and velocities (48), and the previous action (24). It outputs joint position targets as offsets from a default pose, scaled per joint.

Training randomization covers observation noise, initial pose and velocity, velocity pushes every 1 to 3 s, torso center-of-mass offset, joint encoder bias, and foot friction in [0.3, 1.2]. The delay variants add a command delay drawn uniformly from 0 to 50 ms (0 to 10 physics steps) per robot, redrawn every second with staggered phase. The same task exists in both engines. In Isaac Lab we add the R1 to `whole_body_tracking` with a URDF generated from the same MJCF.

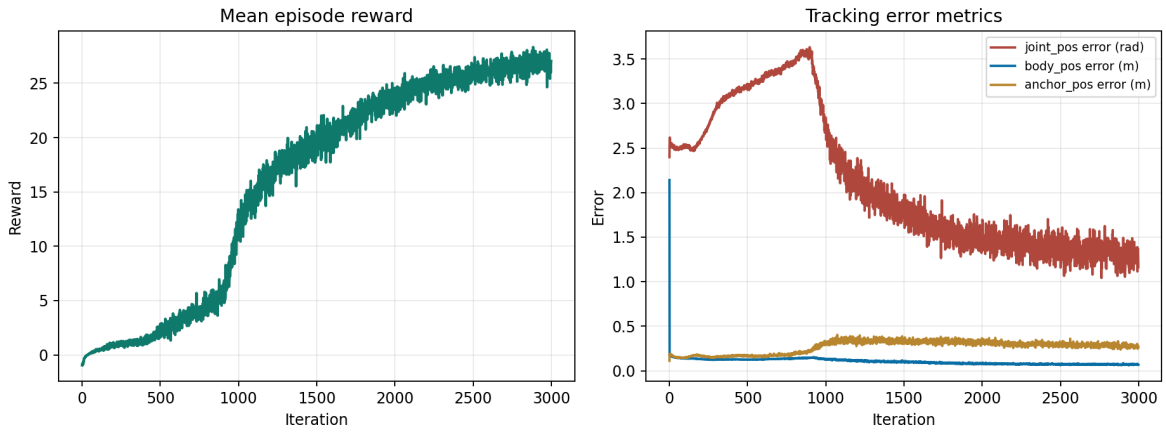


Figure 2: Tennis policy, first 3,000 training iterations in mjlab (59 minutes). Left: mean episode reward. Right: tracking errors. Joint error is the L2 norm over the 24 joints.

5.4 Stage 4: sim2sim harness

The harness runs exported ONNX policies in standalone CPU MuJoCo (`mj_step`). It builds the physics model from the raw MJCF with MuJoCo's spec API and sets the training model's floor, foot contact parameters, timestep, solver iterations and integrator explicitly. It reimplements the observation, the training randomization and the training termination rules: a deviation of the anchor orientation (the vertical component of the gravity direction in the anchor frame, a dimensionless quantity, differing by more than 0.8) or of the anchor height or an end-effector height (more than 0.25 m) ends the episode. For Isaac-trained policies, a name-based mapping translates between PhysX and MuJoCo joint order. A validation mode checks that the harness's first observation equals the one Isaac Lab computes; the maximum difference is 0.00000.

On top of the training randomization the stress scenarios add: a constant command delay of 1 to 3 control steps (20 to 60 ms), pushes at 2 to 3 times the training magnitude, ±20% PD gain, a 2 kg torso payload, foot friction fixed at 0.15, and removal of base position and/or base linear velocity from the observation. There are 17 scenarios in the harness (nominal, the training randomization, and 15 perturbations). Each perturbed scenario runs 20 to 30 seeds; not every scenario was run for every policy. A seed *survives* if it completes the whole clip without triggering a termination rule.

6 Reproducibility

Code, the intermediate data of each stage, five trained policies with their exact configurations, and all raw results are public in the lab's repository (see Code, data and supplementary videos at the end). Upstream systems are pinned: GVHMR `ee960bb`, GMR `bb1bbe4`, mjlab `c2e1e06`, Isaac

Lab `b0542fe2d`, `whole_body_tracking` `cd65172`. On 2026-10-08 we re-ran every stage from fresh clones, using only the released files, and compared against the outputs used in this paper.

Table 4: Reproduction check. Full-length training and Isaac Lab training were not re-run from scratch.

Step	Result
Stage 1, GVHMR on tennis	every output array identical (max absolute difference 0.0)
Stage 2, retarget to R1	root position, rotation and all 24 joints identical
trajectory to CSV	byte-identical
CSV to 50 Hz reference	all 7 arrays identical
Stage 3, both R1 tasks	train and export (5-iteration smoke test; full training is not bit-deterministic on GPU)
Stage 4, delay-trained tennis policy	survival and error identical, seed for seed
Stage 4, Isaac-trained dance policy	observation check 0.00000; joint error identical

7 Results

Two motions: tennis, a serve (10.4 s), and dance (a 26.5 s clip). *Corrected 2026-10-14:* GMR's GVHMR loader assumes 30 fps, so the dance reference was retargeted 1.2× too fast (26.5 s of dancing performed in 22.1 s). All dance results below are for that sped-up reference. The pipeline now takes the video's real frame rate. Five policies:

Table 5: The five policies. All runs used 4,096 parallel robots.

Policy	Engine	Training	In-engine evaluation
tennis	mjlab	3,000 iterations (59 min), continued to 12,998 (+3 h 19 min)	96.9% at 3,000, 98.4% at 12,998 (64 episodes)
tennis + delay	mjlab	fine-tuned from 12,998 for 3,000 iterations (55 min)	not run
dance	mjlab	3,000 iterations	100% (64 episodes)
dance	Isaac Lab	3,000 iterations (about 100 min)	not run
dance + delay	Isaac Lab	3,000 iterations, from scratch	not run

Joint error below is the L2 norm of the 24-joint error vector, averaged over the episode. This is the same quantity as mjlab's `error_joint_pos`. Dividing by $\sqrt{24}$ gives a rough per-joint figure (0.12 rad for tennis, 0.46 rad for the mjlab dance policy). The per-joint RMS computed directly over the episode is higher for the dance, 0.55 rad, because a few joints carry most of the error.

7.1 The first sim2sim failure was the harness

The first harness loaded the robot MJCF directly. The policy diverged: mean joint error 4.6 to 8.9 rad across harness versions. Running the raw PyTorch checkpoint on CUDA instead of ONNX on CPU changed nothing (4.70 rad). Our first diagnosis was a mismatch between the MuJoCo-Warp and native MuJoCo solvers. That was wrong. Ten thousand more training iterations nearly

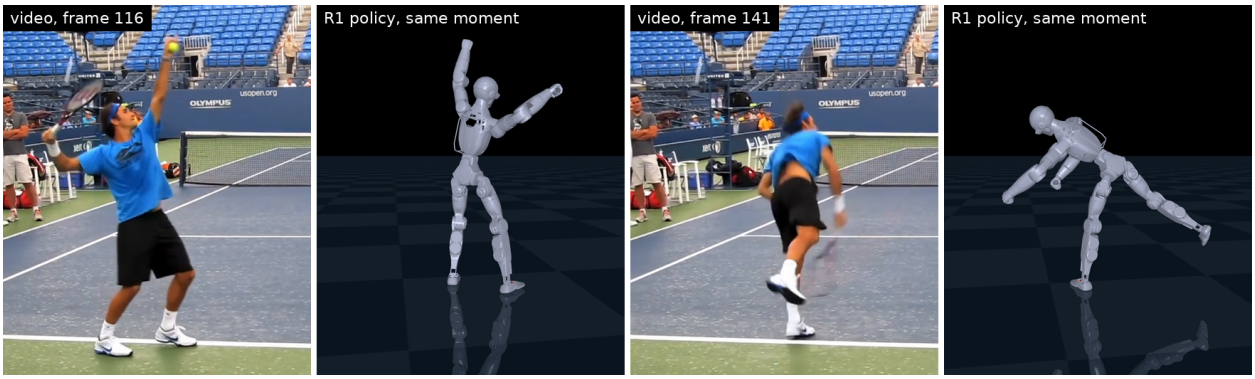


Figure 3: The tennis serve. Left of each pair: the source video frame (30 fps, frames 116 and 141, ball toss and follow-through). Right: the trained tennis policy running in plain MuJoCo at the matching moment. These are policy rollouts, not kinematic replays. The video is GVHMR's example clip (Table 2). Video: `results/videos/r1_tennis_sim2sim_trainDR.mp4`, which completes the clip.

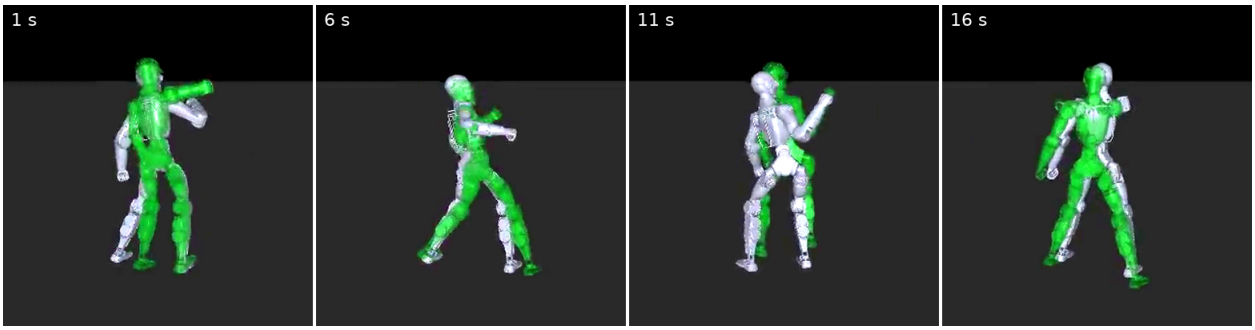


Figure 4: The dance policy (mjlab) running in plain MuJoCo, at 1, 6, 11 and 16 s of the rollout. White is the policy-controlled robot, green is the reference motion. The reference is drawn at its own world position. The robot does not hold that position exactly (mean anchor-position error at the end of training is about 0.25 m), so the two figures sit apart. The source dance clip is not shown. Video: `results/videos/r1_dance_mjlab_in_mujoco.mp4`.

halved the in-engine body-position error (MPKPE 0.30 to 0.16 m) but improved the harness by only 4% (4.79 to 4.59 rad), which pointed at the harness. The raw MJCF has no ground plane, and the harness stepped physics at 125 Hz instead of training's 200 Hz. We rebuilt the training model in the harness and put each raw setting back one at a time:

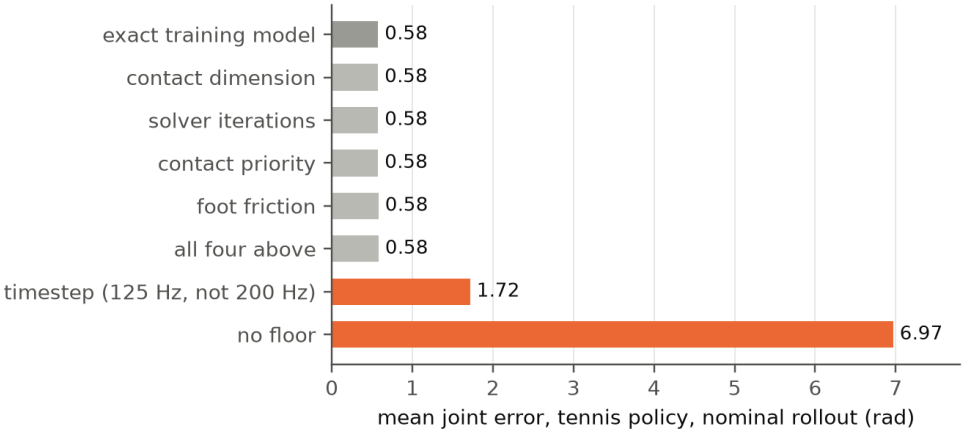


Figure 5: One-at-a-time ablation, tennis policy, nominal rollout. Grey: settings that do not matter. Orange: the two that do. Source: `results/terminal_captures/floor_timestep_ablation.txt`.

Only the floor and the timestep matter. With both matched, the policy tracks the whole clip at 0.577 rad. Contact dimension, solver iterations, foot friction and contact priority each move the error by under 0.006 rad, alone or together.

7.2 Robustness under randomization and stress

Under the randomization they were trained with, every policy trained without delay survives 29 to 30 of 30 seeds, whichever engine trained it. Parameter perturbations within plausible hardware variation (gains ±20%, a 2 kg payload) do no harm. Three perturbations do: command delay, low foot friction, and losing base linear velocity.

7.3 Command delay

Delay is the sharpest failure. 20 ms is tolerated, but at 40 ms only 2 of 20 tennis rollouts survive. Most fail on end-effector height, meaning a foot or hand ends up more than 0.25 m from its reference height. The PhysX-trained dance policy fails the same way and worse, 0 of 20. Since the two policies come from different engines, this is not a quirk of either engine.

Training with a random delay of 0 to 50 ms moves 40 ms survival from 2/20 to 15/20 (mjlab, tennis) and from 0/20 to 15/20 (Isaac, dance). The mjlab policy pays nothing for it under the training randomization (30/30) and becomes slightly less precise (joint error 0.577 to 0.597). The Isaac policy pays heavily: 29/30 drops to 17/30 with no added delay. We do not know why. There are two candidates. The mjlab policy was fine-tuned from a converged checkpoint and the Isaac policy was trained from scratch. And 3,000 iterations may simply not be enough for the harder problem.

One caveat applies to this single comparison, and it matters. The delay-trained tennis policy had 3,000 more training iterations than the one it is compared with. The size of the effect (2/20 to

Table 6: Survival: rollouts that complete the whole clip, out of the number of seeds. Dashes mark scenarios that were not run. Delay is a constant extra delay of 1, 2 or 3 control steps of 20 ms.

Scenario	tennis	tennis + delay	dance (mjlab)	dance (Isaac)	dance (Isaac) + delay
nominal	1/1	1/1	1/1	1/1	1/1
training randomization	30/30	30/30	29/30	29/30	17/30
+ 20 ms delay	19/20	20/20	–	–	18/20
+ 40 ms delay	2/20	15/20	–	0/20	15/20
+ 60 ms delay	0/20	8/20	–	–	11/20
+ 2× pushes	17/20	–	–	–	–
+ 3× pushes	11/20	11/20	–	–	–
+ foot friction 0.15	5/20	10/20	–	–	–
PD gain −20%	20/20	19/20	–	–	–
PD gain +20%	20/20	–	–	–	–
+ 2 kg payload	20/20	20/20	–	–	–
no base position	20/20	–	–	–	–
no base linear velocity	5/20	4/20	–	–	–
no base position or velocity	14/20	–	–	–	–
joint error, nominal (rad)	0.577	0.597	2.24	2.01	2.19

Source: `results/sweeps/*.json` and `results/terminal_captures/dr_sweep_old_policy.txt`. Every count was recomputed from the raw files for the 2026-10-11 manuscript.

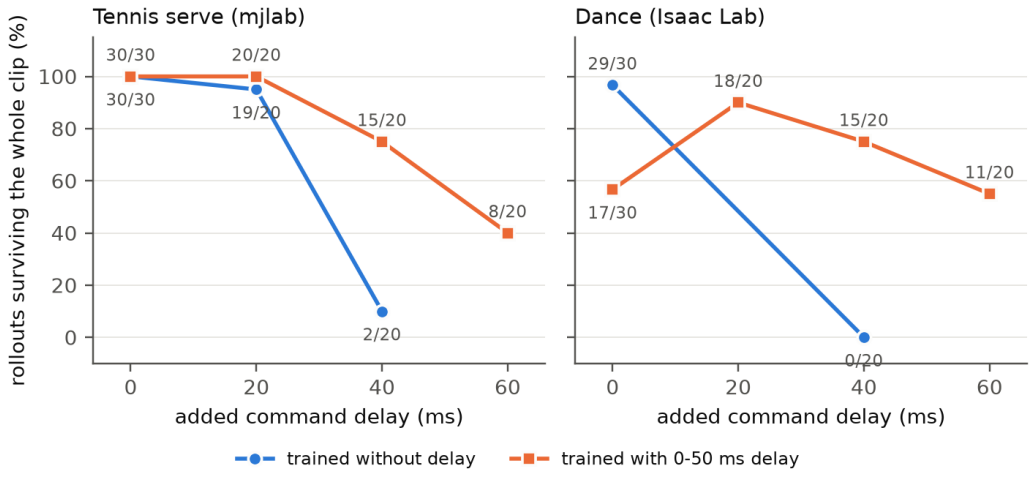


Figure 6: Survival against added command delay. Labels give surviving rollouts over seeds. The tennis policy is from mjlab, the dance policy from Isaac Lab, so the same weakness shows up in both engines. The delay-trained tennis policy was fine-tuned from the no-delay one; the delay-trained dance policy was trained from scratch.

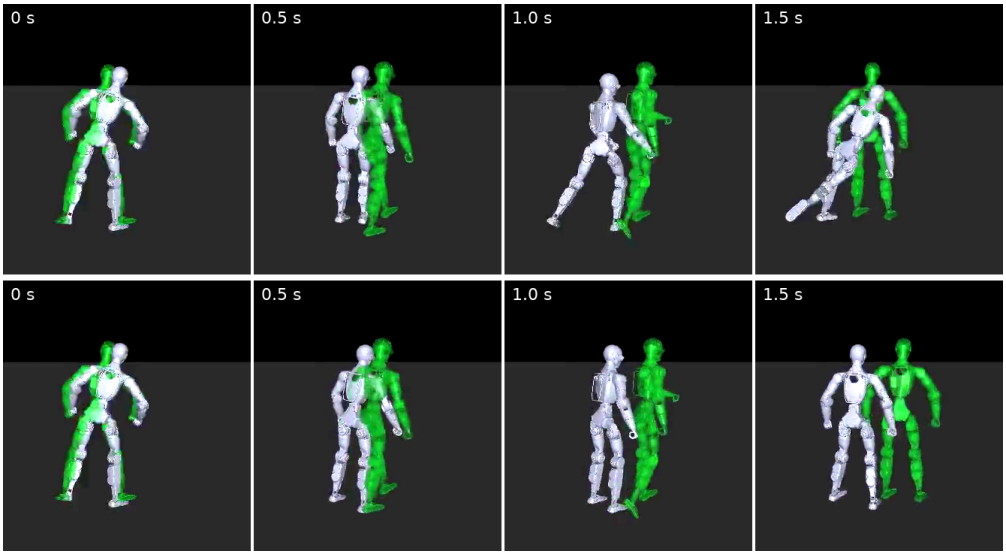


Figure 7: Tennis, 40 ms added delay. Top: the policy trained without delay has lost its balance and is falling by 1.5 s. Bottom: the delay-trained policy stays up. White is the robot, green the reference. Videos: `r1_tennis_sim2sim_latency2_FAIL.mp4` and `r1_tennis_sim2sim_latency2_delaytrained.mp4`.

15/20) and its appearance in both engines make delay the likely cause, and the next subsection tests it directly. The Isaac pair (both trained for 3,000 iterations) does not have this confound.

7.4 Seed study: is the gain from delay or from more training?

The mjlab delay policy was fine-tuned for 3,000 extra iterations, so its improvement could come from the extra training rather than from the delay. To separate the two, we continued the same tennis checkpoint (12,998 iterations) for 3,000 iterations *without* delay (control), using three training seeds per arm. The delay arm reuses the existing run as seed 42. All six policies were then swept in the harness.

Table 7: Seed study, tennis, mjlab. Survival per training seed (42 / 43 / 44), out of the number in parentheses.

Scenario	control: +3,000 iterations, no delay	delay: +3,000 iterations, 0–50 ms delay
training randomization (30)	30 / 30 / 30	30 / 30 / 30
+ 20 ms delay (20)	19 / 19 / 20	20 / 20 / 20
+ 40 ms delay (20)	**4 / 8 / 2**	**15 / 20 / 20**
+ 60 ms delay (20)	**0 / 0 / 0**	**8 / 12 / 14**
+ 3× pushes (20)	12 / 13 / 13	11 / 17 / 10
+ foot friction 0.15 (20)	5 / 3 / 6	10 / 9 / 10
joint error, nominal (rad)	0.566 / 0.552 / 0.555	0.597 / 0.632 / 0.588

Extra training alone leaves 40 ms survival at 2–8/20 (mean 4.7), close to the original policy's 2/20. With delay training it is 15–20/20 (mean 18.3), and at 60 ms 8–14/20 against 0/20 for every control seed. **The gain comes from the delay randomization, not from the additional**

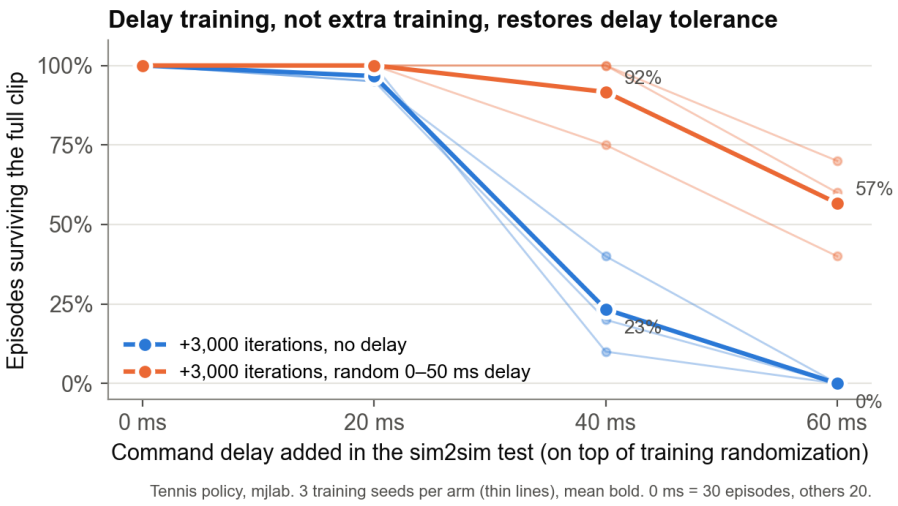


Figure 8: Survival against added delay for the three training seeds of each arm (thin lines) and their means (bold). Tennis policy, mjlab. Zero added delay is out of 30 episodes, the others out of 20.

iterations. It costs nothing under the training randomization (30/30 in all six runs), and costs a small amount of precision: nominal joint error is 0.59–0.63 against 0.55–0.57. One side effect we did not expect: survival at foot friction 0.15 roughly doubles (mean 4.7 to 9.7 of 20), while survival under 3× pushes is unchanged (mean 12.7 in both arms). Seed-to-seed spread reaches 7 of 20 episodes (pushes, delay arm), so single-seed differences smaller than that should not be read as real. Raw results: `results/sweeps/seed_study/`; policies: `policies/seed_study/`.

7.5 Base linear velocity

Removing base position from the observation is harmless (20/20). Removing base linear velocity leaves 5/20, and 4/20 after delay training. Removing both gives 14/20, which is better than velocity alone. That is consistent with the policy using the two signals together, but we did not test it further. A real R1 has no direct measurement of base linear velocity. Before hardware, either a state estimator must provide it or the policy must be trained without it. mjlab's no-state-estimation task variant, included in the release, does the latter. We have not evaluated it.

8 Discussion

8.1 Check the harness before blaming the physics

A sim2sim check says it compares two simulators. In practice it compares two models. Our first harness differed from training in two settings that someone reading the MJCF would not see: the floor comes from the training framework's terrain and not from the robot file, and the timestep comes from the task configuration. The ablation in Figure 5 is cheap, and it turns a vague "physics gap" into two named, fixable causes. We suggest running it before reporting any sim2sim gap.

8.2 Delay is the gap that sim2sim checks leave out

A standard sim2sim check steps both simulators synchronously, so it cannot show sensitivity to delay unless delay is put in on purpose. Here 40 ms was enough to break policies from both engines. Our earlier R1 walking study (internal) saw the same thing from the other side: with a fixed delay of one control step (20 ms) a walk that lasted 20 s lasted about 5 s, and with two steps (40 ms) 2.7 s. Real control loops routinely have tens of milliseconds between policy and actuator. The first number to measure on hardware is the actual command latency, and the training delay range should be centred on it.

8.3 Joint-space error misleads on video-derived references

The dance policy's joint error (2.24 rad) is nearly four times the tennis policy's (0.58), yet it completes the clip in both engines. This is not a harness artifact: mjlab's own training metric ends at 2.04. Per joint, the error sits almost entirely in the left shoulder (pitch RMS 1.49 rad, yaw 1.31 rad, with signed means of +1.03 and +0.89 rad). In 27% of dance reference frames the left shoulder is pinned within 0.02 rad of a joint limit, almost all of it in shoulder yaw (25%) and roll (19%). In those frames the left-shoulder error is 2.06 rad RMS, against 0.45 rad in the others. The left hand, though, lands almost as well in both sets (19 cm against 16 cm, measured in the world frame, so including root drift). The policy reaches the pose through a different shoulder configuration than the one the IK settled on when it clamped at the limit.

Retargeting reports these frames as having no limit violations, because IK clamps at the limit. We counted how often the reference sits at a limit more generally, using the released motion files and a tolerance of 0.001 rad. On the dance reference at least one joint is at a limit on 62% of frames (right ankle roll 36%, left shoulder yaw 25%, left shoulder roll 18%). On the tennis reference it is 31% (right ankle roll 19%, left shoulder yaw 16%, waist roll 9%). A violation count of zero should not be read as "this reference is comfortably feasible". Joint-space error on such references mixes real tracking failure with harmless redundancy, and task-space error, or a count of how often the reference sits at a limit, tells you more. The dance policy also had a quarter of the tennis policy's training iterations, which explains part of the gap.

9 Limitations

- **Simulation only.** All results use idealized actuators (PD with torque limits, no motor dynamics). The sim2real items raised here (delay, base velocity, friction margin) have not been tested on a physical R1.
- **Two third-party, static-camera clips.** Three training seeds only for the mjlab delay result (Section 7.4); every other policy is a single seed.
- The harness holds delay constant for the whole episode. Training redraws it every second and starts at zero, so the 40 to 60 ms scenarios are stricter than training.
- Isaac Lab training ran on native Windows because GPU PhysX hangs under WSL2 on this GPU. The harness ran under WSL2. The two engines' policies are compared in the same harness, but their training runs differ in more than the engine (from scratch against fine-tuned).
- The cost of delay training in PhysX (29/30 to 17/30) is unexplained and is a single run.
- The dance reference was retargeted 1.2× too fast (Section 7); the dance results are for that reference and have not been re-run at the real frame rate.

- The dance clip is not redistributed, so that result cannot be re-run without getting the clip from Pexels. Stage 1 requires the GVHMR checkpoints and the SMPL-X model, which users must download themselves.

10 Future work

Three things would turn this from a simulation study into a deployment study. Our own handheld video, to test what the two third-party clips cannot: a moving camera (GVHMR's visual-odometry path), our own lighting and clothing, and whether the ground-drift correction survives a messier recovery. The physical R1. The results set the order: measure the real command latency and retrain with the delay range centred on it; supply base linear velocity from an estimator or train the no-state-estimation variant; check the friction margin of the lab floor; then deploy supported on a gantry before free-standing. Isaac Lab seeds, to settle whether the cost of delay training in PhysX (29/30 to 17/30) is real or a single-run effect, and a re-run of the dance policy at the corrected frame rate.

11 Conclusion

In simulation, a video of a person can be turned into a humanoid policy for a robot that the upstream tools we used did not support, using open tools on one workstation, and each stage can be re-run and checked. What the resulting policies lacked was not physics fidelity. They lacked robustness to things a synchronous sim2sim check does not contain: command delay, an unobservable base velocity, and low friction. Two of our three findings are about the check and not the policy: the harness must match the training model's floor and timestep, and joint-space error must be read against how often the reference sits at a joint limit. Whether these findings carry over to the physical R1 is the next question.

Code, data and supplementary videos

Everything is in one public folder: https://github.com/SunCCNY/Embodied-AI/tree/main/Presentations/Shaoqin-Li/Video-to-R1%20Pipeline. It holds the code for all four stages, the GVHMR and GMR outputs for both clips, five policies (ONNX, final checkpoint, configuration), the six seed-study policies, the sweeps as JSON, terminal captures, and the videos. Not included: the SMPL-X model (license), the GVHMR checkpoints (size), and the dance source video (third party). Experiments ran on one workstation with an NVIDIA RTX 5080 GPU.

Videos referenced in the figures, all in `results/videos/`: `r1_tennis_sim2sim_trainDR.mp4`, `r1_tennis_sim2sim.mp4` (the original harness without a floor and at 125 Hz, Section 7.1), `r1_tennis_sim2sim_latency2_FAIL.mp4`, `r1_tennis_sim2sim_latency2_delaytrained.mp4`, `r1_dance_mjlab_in_mujoco.mp4`, `r1_dance_isaac_in_mujoco.mp4`, `r1_dance_isaac_latency2_nodelay.mp4`, `r1_dance_isaac_latency2_delaytrained.mp4`. In every video grey or white is the policy-controlled robot and green is the reference.

References

[1] Allshire, A., Choi, H., Zhang, J., McAllister, D., Zhang, A., Kim, C. M., Darrell, T., Abbeel, P., Malik, J., Kanazawa, A. Visual Imitation Enables Contextual Humanoid Control (VideoMimic). arXiv:2505.03729, 2025.

[2] He, T., Luo, Z., Xiao, W., Zhang, C., Kitani, K., Liu, C., Shi, G. Learning Human-to-Humanoid Real-Time Whole-Body Teleoperation (H2O). arXiv:2403.04436, 2024.

[3] He, T., Gao, J., Xiao, W., Zhang, Y., Wang, Z., Wang, J., Luo, Z., He, G., Sobanbab, N., Pan, C., Yi, Z., Qu, G., Kitani, K., Hodgins, J., Fan, L. J., Zhu, Y., Liu, C., Shi, G. ASAP: Aligning Simulation and Real-World Physics for Learning Agile Humanoid Whole-Body Skills. RSS 2025. arXiv:2502.01143.

[4] Liao, Q., Truong, T. E., Huang, X., Gao, Y., Tevet, G., Sreenath, K., Liu, C. K. BeyondMimic: From Motion Tracking to Versatile Humanoid Control via Guided Diffusion. arXiv:2508.08241, 2025.

[5] Unitree Robotics. Unitree R1 (product page, accessed 2026-10-11). https://www.unitree.com/R1

[6] He, T., Luo, Z., He, X., Xiao, W., Zhang, C., Zhang, W., Kitani, K., Liu, C., Shi, G. OmniH2O: Universal and Dexterous Human-to-Humanoid Whole-Body Teleoperation and Learning. arXiv:2406.08858, 2024.

[7] Xie, W., Han, J., Zheng, J., Li, H., Liu, X., Shi, J., Zhang, W., Bai, C., Li, X. KungfuBot: Physics-Based Humanoid Whole-Body Control for Learning Highly-Dynamic Skills. NeurIPS 2025. arXiv:2506.12851.

[8] Chen, Z., Ji, M., Cheng, X., Peng, X., Peng, X. B., Wang, X. GMT: General Motion Tracking for Humanoid Whole-Body Control. arXiv:2506.14770, 2025.

[9] Ze, Y., Chen, Z., Araujo, J. P., Cao, Z., Peng, X. B., Wu, J., Liu, C. K. TWIST: Teleoperated Whole-Body Imitation System. arXiv:2505.02833, 2025.

[10] Amadio, F., Mingo Hoffman, E. What Matters in Humanoid General Motion Tracking? An Empirical Study. arXiv:2607.19903, 2026.

[11] Pavlakos, G., Choutas, V., Ghorbani, N., Bolkart, T., Osman, A. A. A., Tzionas, D., Black, M. J. Expressive Body Capture: 3D Hands, Face, and Body from a Single Image (SMPL-X). CVPR 2019.

[12] Shen, Z., Pi, H., Xia, Y., Cen, Z., Peng, S., Hu, Z., Bao, H., Hu, R., Zhou, X. World-Grounded Human Motion Recovery via Gravity-View Coordinates (GVHMR). SIGGRAPH Asia 2024. arXiv:2409.06662.

[13] Araujo, J. P., Ze, Y., Xu, P., Wu, J., Liu, C. K. Retargeting Matters: General Motion Retargeting for Humanoid Motion Tracking. arXiv:2510.02252, 2025.

[14] Yang, L., Huang, X., Wu, Z., Kanazawa, A., Abbeel, P., Sferrazza, C., Liu, C. K., Duan, R., Shi, G. OmniRetarget: Interaction-Preserving Data Generation for Humanoid Whole-Body Loco-Manipulation and Scene Interaction. arXiv:2509.26633, 2025.

[15] Luo, Z., Cao, J., Winkler, A., Kitani, K., Xu, W. Perpetual Humanoid Control for Real-time Simulated Avatars (PHC). ICCV 2023. arXiv:2305.06456.

[16] Gu, X., Wang, Y.-J., Chen, J. Humanoid-Gym: Reinforcement Learning for Humanoid Robot with Zero-Shot Sim2Real Transfer. arXiv:2404.05695, 2024 (ICRA 2024 Workshop on Agile Robotics).

[17] Lei, Z., Zhou, Z., Yin, S., Chen, Y., Xu, Q., Li, W., Wang, Y., Tang, B., Jing, W., Chen, S. PolySim: Bridging the Sim-to-Real Gap for Humanoid Control via Multi-Simulator Dynamics Randomization. arXiv:2510.01708, 2025.

[18] Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., Abbeel, P. Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World. IROS 2017.

[19] Peng, X. B., Andrychowicz, M., Zaremba, W., Abbeel, P. Sim-to-Real Transfer of Robotic Control with Dynamics Randomization. ICRA 2018.

[20] Hwangbo, J., Lee, J., Dosovitskiy, A., Bellicoso, D., Tsounis, V., Koltun, V., Hutter, M. Learning agile and dynamic motor skills for legged robots. Science Robotics 4(26), 2019.

[21] Li, S. What a sim2sim check can and cannot tell you. Internal working paper, Applied and Embodied AI Laboratory, CCNY, 2026 (unpublished).

[22] Ze, Y., et al. GMR: General Motion Retargeting (software, MIT license). https://github.com/YanjieZe/GMR

[23] Zakka, K., Liao, Q., Yi, B., Le Lay, L., Sreenath, K., Abbeel, P. mjlab: A Lightweight Framework for GPU-Accelerated Robot Learning. arXiv:2601.22074, 2026.

[24] Mittal, M., et al. (NVIDIA). Isaac Lab: A GPU-Accelerated Simulation Framework for Multi-Modal Robot Learning. arXiv:2511.04831, 2025.

[25] HybridRobotics. whole_body_tracking (BeyondMimic motion-tracking code, software). https://github.com/HybridRobotics/whole_body_tracking

[26] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O. Proximal Policy Optimization Algorithms. arXiv:1707.06347, 2017.

[27] Schwarke, C., Mittal, M., Rudin, N., Hoeller, D., Hutter, M. RSL-RL: A Learning Library for Robotics Research. arXiv:2509.10771, 2025.

[28] Todorov, E., Erez, T., Tassa, Y. MuJoCo: A physics engine for model-based control. IROS 2012.

[29] Unitree Robotics. unitree_mujoco (software, BSD-3-Clause). https://github.com/unitreerobotics/unitree_mujoco

[30] Shkraba, A. Woman Dancing (stock video). Pexels. https://www.pexels.com